

case you are using a remote database as you may be sending passwords in clear text over the network! I'm using a local database over a UNIX socket so it doesn't matter.

The database structure for this is as follows:

```
CREATE TABLE `http_users` (
  `username` varchar(15) NOT NULL DEFAULT '',
  `password` varchar(56) NOT NULL DEFAULT '',
  `groups` set('superadmin','group1','group2','group3','group4') NOT NULL DEFAULT '',
  PRIMARY KEY (`username`),
  UNIQUE KEY `password` (`password`)
) ENGINE=InnoDB DEFAULT CHARSET=latin1;

DELIMITER $$
CREATE trigger http_user_ins before insert on http_users for
each row
begin
SET NEW.password = SHA2(NEW.password, 224);
end;
DELIMITER $$

DELIMITER $$
CREATE trigger http_user_upd before update on http_users for
each row
begin
if LENGTH(NEW.password) != 56 then
SET NEW.password = SHA2(NEW.password, 224);
end if;
end;
DELIMITER $$
```

The triggers are required to convert the INSERT or UPDATE statements into SHA224. I've used MySQL's SET data type to ensure that the group value cannot contain errors - that is, it can only have a fixed set of values. This gets even better when you use PHPMyAdmin - it will give a list of check boxes for each of the group values. The same values can be used by Nginx in the `$user_group` variable before specifying the `access_by_lua_file` directive.

However, there is one problem with this method of securing access. This is that if you embed a location statement inside the location block in which you are calling the authentication script, as shown below (which happens to be a common pattern with modern PHP Web applications)...

```
location ~ ^/restricted {
    set $user_group 'somegroup';
    access_by_lua_file '/etc/nginx/authenticate.lua';

    location ~ ^{<script_name>.+\.php}{<path_info>/.*}$ {
        try_files $script_name =404;
        fastcgi_pass unix:/var/run/php-fpm.sock;
        include fastcgi_params;
    }
}
```

```
}
try_files $uri $uri/ /index.php$is_args$args;
}
```

...then the authentication process works only for the path `/restricted`; it doesn't work for the paths under the tree such as `/restricted/a`. I haven't been able to find the exact reason to this but I have a naive fix for it:

```
local auth_groups = {
    ["group1"] = {"/restricted"};
    ["group2"] = {"/restricted2"};
}

local user_group

for groupname, urls in pairs(auth_groups) do
    for _, url in pairs(urls) do
        if ngx.var.request_uri:find(url) then
            user_group = groupname
            break
        end
    end
end
```

Run this piece of code after the basic configuration of the authentication script. This is naive because it does a linear (sequential) search in all the groups for every request. If you are going to use this on a large scale, a linear search can quickly eat up your CPU resources.

This piece of code changes the Nginx part of the configuration slightly. We were earlier sending `$user_group` from Nginx; now we don't have to, as the Lua script will verify the URL to group mapping, as follows:

```
location /phpMyAdmin {
    root /usr/local/www;
    access_by_lua_file 'authenticate.lua';

    location ~ ^{<script_name>.+\.php}{<path_info>/.*}$ {
        try_files $script_name =404;
        fastcgi_pass unix:/var/run/php-fpm.sock;
        include fastcgi_params;
    }
}
```

The authentication script is available on my GitHub account: <https://github.com/nileshgr/utilities/blob/master/admin/nginx-group-authenticate.lua>. 

By: Nilesh Govindrajana

The author is a technology enthusiast based in Pune, India. He can be contacted at @nileshgr on Twitter or emailed at me@nileshgr.com.